# 《操作系统综合实验》 实验报告

| 实验名称 | 系统调用基础 | 实验序号 | 1 |
| --- | --- | --- | --- |
| 实验日期 | 2024.3.23 | 实验人 | 廖鑫洋 |

## 一、 实验题目

在 2.3 节中，我们描述了一个将一个文件的内容复制到目标文件的程序。该程序首先提示用户输入源文件和目标文件的名称。使用 Windows 或 POSIX API 编写此程序。请务必包括所有必要的错误检查，包括确保源文件是否存在。

正确设计和测试程序后，如果你使用支持它的系统，请使用跟踪系统调用的实用程序运行该程序。 Linux 系统提供 strace 实用程序，Solaris 和 Mac OS X 系统使用 dtrace 命令。由于 Windows 系统不提供此类功能，您将不得不使用调试器跟踪此程序的 Windows 版本。

## 二、 相关原理与知识

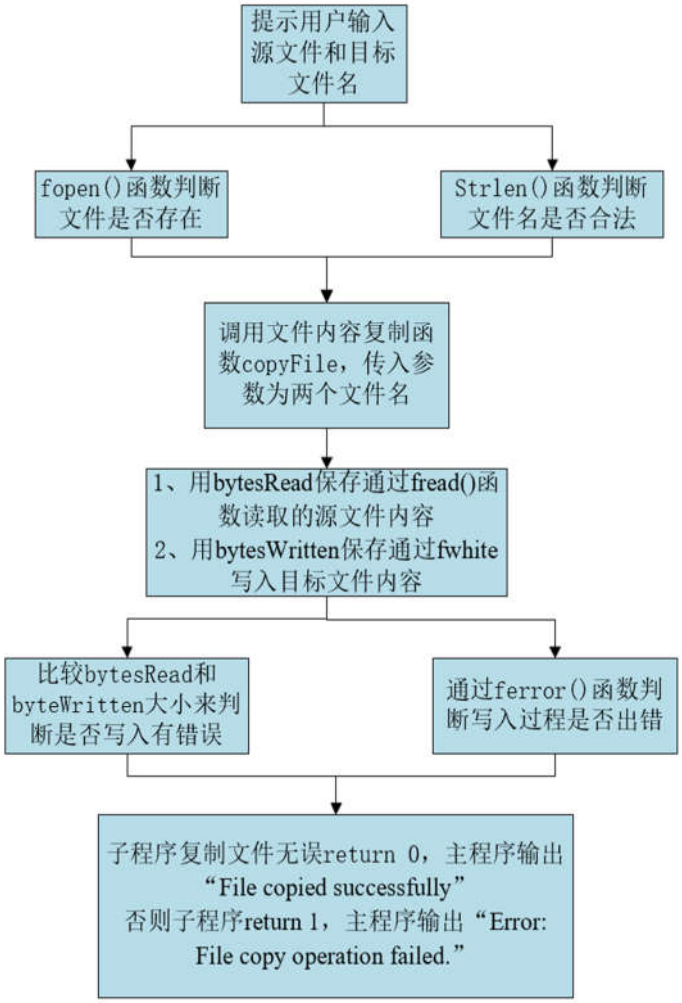
1、文件系统，是操作系统存储和组织文件的一种方式，定义文件存储结构和访问方式。文件系统通常提供了删除、创建、读取、写入等机制。本题关于文件复制，需要操作系统定位源文件和目标文件，然后根据文件系统来对两个文件进行相关操作。

2、系统调用，是程序向操作系统请求服务的一种方式。当程序需要执行如文件操作这样的低级任务时，它会通过系统调用来请求操作系统完成这些任务。在文件复制的例子中，系统调用如 open、read、write 和 close 被用来打开文件、读取内容、写入内容和关闭文件。

3、错误检查和异常处理。在文件操作过程中，可能会遇到各种错误情况，如文件不存在、权限不足、磁盘空间不足等。操作系统提供了错误码和异常处理机制，使得程序能够检测到这些错误并作出相应的处理。

## 三、 实验过程

本实验代码设计过程，主要涉及到用 c 语言进行读写操作，代码设计思路按照 c 语言面向过程来实现，同时需要考虑输入提醒、各种异常情况的发生。主要设计思路流程如下流程图所示。




## 四、 实验结果与分析

1）gcc -o main main.c

用于编译 main.cC 语言源代码的 GCC 命令，该命令行将对 main.c 文件进行编译生成可执行文件 main。

-o 指定输出文件的名称为 main。

2）strace ./main

执行同级文件夹下的 main 的可执行文件，通过调用 Linux 的 strace 来跟踪显示 main 执行过程中的系统调用，输出到终端窗口。

3）执行情况解释

1、strace: Symbol '_uPT_accessors' has different size in shared object, consider re-linking 表示 strace 检测到共享对象中的 _uPT_accessors 符号大小不一致，建议重新链接程序。

2、execve(“./main”, [“./main”], 0x7fff48cf2710 /* 57 vars */) = 0 表示 main 程序开始执行。

3、一系列 map 调用，如 map(NULL, 84262, PROT_READ, MAP_PRIVATE, ...)，表示操作系统为程序分配内存并映射共享库到进程的地址空间。

4、openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC)表示打开 /etc/ld.so.cache 文件。

5、close(3) 关闭文件描述符 3。

6、标准输入输出操作，write(1, "Enter the source file name: ", 28) 向标准输出（通常是终端）写入提示信息。

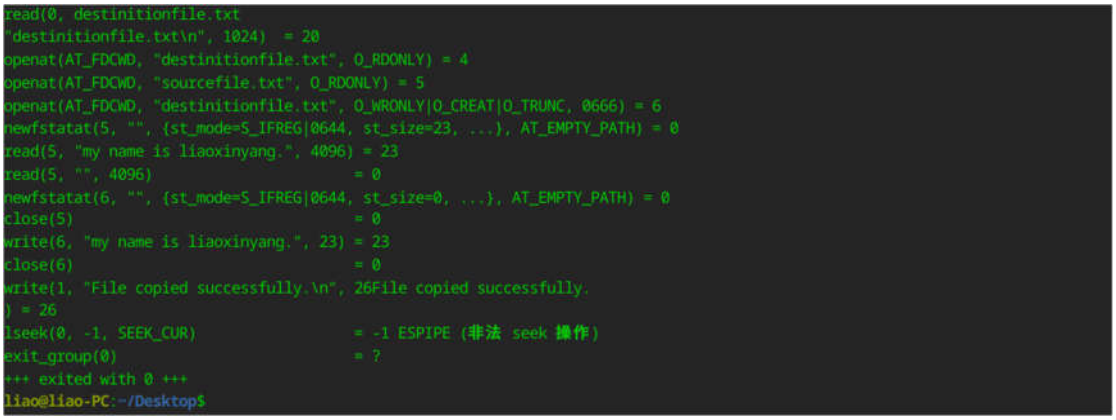
7、read(3, ex5596763f26be, 1024) 从文件描述符 3（可能是标准输入）读取数据，但遇到了 ERESTARTSYS 错误，这通常意味着系统调用被信号中断，需要重新启动。

8、read(0, )第一个光标停止等待输入源文件名称，第二个光标停止表示等待输入目标文件名称。

9、exit_group(0)：用于终止当前进程以及同一进程组中的所有其他进程。表示系统调用结束。

```
liao@liao-PC:~/Desktop$ gcc -o main main.c
liao@liao-PC:~/Desktop$ strace ./main
strace: Symbol `_UPT_accessors' has different size in shared object, consider re-linking
execve("./main", ["./main"], 0x7ffe4dda13a0 /* 57 vars */) = 0
brk(NULL)                               = 0x55e090da9000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f213c089000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (没有那个文件或目录)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=84262, ...}, AT_EMPTY_PATH) = 0
mmap(NULL, 84262, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f213c074000
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0Py\2\0\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0"..., 784, 64) = 784
newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=1918064, ...}, AT_EMPTY_PATH) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0"..., 784, 64) = 784
mmap(NULL, 1970000, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f213be93000
mmap(0x7f213beb9000, 1392640, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x26000) = 0x7f213beb9000
mmap(0x7f213c00d000, 344064, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x17a000) = 0x7f213c00d000
mmap(0x7f213c061000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1cd000) = 0x7f213c061000
mmap(0x7f213c067000, 53072, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f213c067000
close(3)                                = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f213be90000
arch_prctl(ARCH_SET_FS, 0x7f213be90740) = 0
set_tid_address(0x7f213be90a10)         = 4866
set_robust_list(0x7f213be90a20, 24)     = 0
rseq(0x7f213be91060, 0x20, 0, 0x53053053) = 0
mprotect(0x7f213c061000, 16384, PROT_READ) = 0
mprotect(0x55e09082d000, 4096, PROT_READ) = 0
mprotect(0x7f213c0bb000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x7f213c074000, 84262)           = 0
newfstatat(1, "", {st_mode=S_IFCHR|0600, st_rdev=makedev(0x88, 0x1), ...}, AT_EMPTY_PATH) = 0
getrandom("\x7a\x9a\x34\x0e\x86\x49\x6b\x2c", 8, GRND_NONBLOCK) = 8
brk(NULL)                               = 0x55e090da9000
brk(0x55e090dca000)                     = 0x55e090dca000
newfstatat(0, "", {st_mode=S_IFCHR|0600, st_rdev=makedev(0x88, 0x1), ...}, AT_EMPTY_PATH) = 0
write(1, "Enter the source file name: ", 28Enter the source file name: ) = 28
read(0, 0x55e090da96b0, 1024)           = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
--- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
read(0, 0x55e090da96b0, 1024)           = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
--- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
read(0, 0x55e090da96b0, 1024)           = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
--- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
read(0, 0x55e090da96b0, 1024)           = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
--- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
read(0, 0x55e090da96b0, 1024)           = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
--- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
read(0, 0x55e090da96b0, 1024)           = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
--- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
read(0, 0x55e090da96b0, 1024)           = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
--- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
read(0, 0x55e090da96b0, 1024)           = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
--- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
read(0, 0x55e090da96b0, 1024)           = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
--- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
read(0, 0x55e090da96b0, 1024)           = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
--- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
read(0, 0x55e090da96b0, 1024)           = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
--- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
read(0, 0x55e090da96b0, 1024)           = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
--- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
read(0, 0x55e090da96b0, 1024)           = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
--- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
read(0, 0x55e090da96b0, 1024)           = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
--- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
read(0, 0x55e090da96b0, 1024)           = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
--- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
read(0,
```

```
read(0, sourcefile.txt
"sourcefile.txt\n", 1024)       = 15
openat(AT_FDCWD, "sourcefile.txt", O_RDONLY) = 3
write(1, "Enter the destination file name:"..., 33Enter the destination file name: ) = 33
read(0, 0x55e090da96b0, 1024)           = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
--- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
read(0, 0x55e090da96b0, 1024)           = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
--- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
read(0, 0x55e090da96b0, 1024)           = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
--- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
read(0,
```




## 五、 问题总结

1、**问题**：需要在 openKylin 上去安装下载跟踪系统调用的 strace，但是 openKylin 本身没有这个库，使用 sudo apt install strace 命令行会报错，查找资料之后也没有办法去解决，老师说需要直接引入 strace 的压缩包然后在 opensKylin 上解压缩就行，但是还是没有尝试这种解决方法。

**解决方法**：换一个 Linux——deepin，做实验之前 openKylin 和 deepin 在虚拟机上都安装了，于是就直接关闭 opensKylin 打开 deepin，发现可以，解决。

2、**问题**：本题核心是用 c 语言对文件读写操作，对这方面还不熟练。

**解决方法**：搜索文件打开、文件读取、文件写入等操作的代码解析。

3、**问题**：在 windows 上编写的 c 语言代码无法转移到 deepin 上，直接拖不动，想到用微信或者 QQ 传文件，下载不了，无果。

**解决方法**：打开网页微信传输助手，解决。

## 六、 源代码

```
#include <stdio.h>
#include <string.h>
int copyFile(const char *source, const char *destination);
```

```
int main()
{
    char sourceFileName[256], destinationFileName[256];

    // 提示用户输入源文件名
    printf("Enter the source file name: ");
    scanf("%255s", sourceFileName);

    // 检查源文件名是否有效
    if (strlen(sourceFileName) == 0) {
        printf("Error: Source file name cannot be empty.\n");
        return 0;
    }

    //输入源文件是否存在
    FILE *sourcefile = fopen(sourceFileName, "r");
    if(sourcefile == NULL){
        printf("Error: Source file does not exist.\n");
        return 0;
    }

    // 提示用户输入目标文件名
    printf("Enter the destination file name: ");
    scanf("%255s", destinationFileName);

    // 检查目标文件名是否有效
    if (strlen(destinationFileName) == 0) {
        printf("Error: Destination file name cannot be empty.\n");
        return 0;
    }
```

```
    //输入目标文件是否存在
    FILE *destinationfile = fopen(destinationFileName, "r");
    if(destinationfile == NULL){
        printf("Error: destination file does not exist.\n");
        return 0;
    }

    // 调用复制文件的函数
    if (copyFile(sourceFileName, destinationFileName) != 0) {
        printf("Error: File copy operation failed.\n");
        return 0;
    }

    printf("File copied successfully.\n");
    return 0;
}

int copyFile(const char *source, const char *destination)
{
    FILE *sourceFile, *destinationFile;
    char buffer[1024];
    size_t bytesRead, bytesWritten;

    sourceFile = fopen(source, "rb");
    destinationFile = fopen(destination, "wb");

    // 循环读取和写入文件内容
    while ((bytesRead = fread(buffer, 1, sizeof(buffer), sourceFile)) > 0) {
        bytesWritten = fwrite(buffer, 1, bytesRead, destinationFile);
```

```
            if (bytesWritten < bytesRead) {
                perror("Error writing to destination file");
                fclose(sourceFile);
                fclose(destinationFile);
                return -1;
            }
        }

        // 检查是否有读取错误
        if (ferror(sourceFile)) {
            perror("Error reading from source file");
            fclose(sourceFile);
            fclose(destinationFile);
            return -1;
        }

        fclose(sourceFile);
        fclose(destinationFile);

        return 0;
}
```